Jump Oriented Programming

Michele LORETI
Università di Camerino





https://cybersecnatlab.it

License & Disclaimer

License Information

This presentation is licensed under the Creative Commons BY-NC License



To view a copy of the license, visit:

http://creativecommons.org/licenses/by-nc/3.0/legalcode

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

© CINI – 2021

Goal

- The goal of this lecture is to present Jump Oriented Programming (JOP)…
  - first the basic principles of JOP will be introduced
  - then relation with ROP are discussed

© CINI – 2021

Prerequisites

- Lecture:
  - Basic knowledge of C
  - Mechanisms of Memory Corruption

© CINI – 2021

Outline

- ROP Limitations
- Jump Oriented Programming
- JOP vs JOP

© CINI – 2021

Countermeasures to ROP

- We have seen that ROP permits executing arbitrary code by using a sequence of *gadgets*
  - These are short sequences of instructions terminating with *ret*
- Some techniques have been proposed to detect ROP attacks based on the observation of
  - execution of small set of instructions ending with *ret*
  - *pop*s of return addresses all pointing at the same specific memory space
- Moreover, to prevent ROP, a *return-less* approach can be used
  - The use of *ret* is not allowed

© CINI – 2021

Jump-Oriented Programming (JOP)

- Jump-Oriented Programming (JOP) is a technique that triggers the execution of a given functions via a sequence of *indirect jump instructions*
- Similarly to ROP, JOP is based on a sequence of small *gadgets…*
  - in ROP each gadget ends with a return instruction (ret)
  - in JOP each gadget ends with an unconditional jump instruction (jmp)

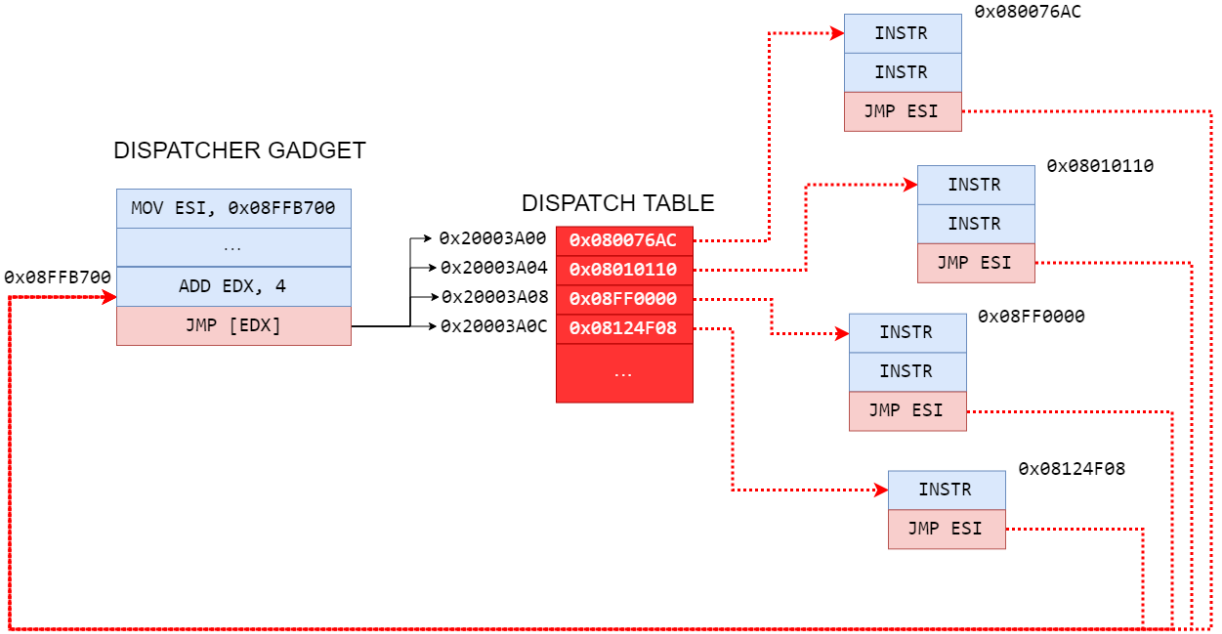
© CINI – 2021

Jump-Oriented Programming (JOP)

- Gadget addresses are collected in a *dispatch table* located in any section of data memory containing a vulnerability (stack, but also heap, global, etc.)
- The final JMP of the gadget is redirected to the *dispatcher gadget*, which advances a pointer to the dispatcher table
- The dispatcher ends with an indirect jump to the address pointed by the pointer, in order to pass the control to the next gadget listed in the table

© CINI – 2021

Jump-Oriented Programming (JOP)



Jump-oriented programming: a new class of code-reuse attack
Tyler Bletsch, Xuxian Jiang, Vincent W. Freeh, Zhenkai Liang
ACM ASIACCS'11

© CINI – 2021 Rel. 07.02.2021

JOP vs ROP

- Both are based on *gadgets*
  - In ROP a gadget terminates with a *ret*
  - In JOP a gadget terminates with a *jmp*
- In ROP the chain of gadgets is stored in the stack
- In JOP each a *dispatcher* that is responsible for *advancing* a *virtual program counter* pointing to the next gadget
- ROP requires control over the instruction pointer *eip* and stack pointer *esp*
- JOP requires *eip* plus a memory location (or registers) to run the dispatcher gadgets

© CINI – 2021

Jump Oriented Programming

Michele LORETI
Università di Camerino

CYBER CHALLENGE.IT

CYBERSECURITY NATIONAL LABORATORY

https://cybersecnatlab.it